

Sau bài thực hành này, sinh viên có thể:

- Cài đặt được giải thuật Linear Regression
- Ứng dụng Linear Regression giải quyết các vấn đề thực tế trên nền tảng Web sử dụng Flask

Trong bài thực hành này, chúng ta sẽ tìm hiểu về học máy cơ bản. Học máy có 3 phương pháp là:

- Học giám sát (Supervised Learning)
- Học không giám sát (Unsupervised Learning)
- Học tăng cường (Reinforcement Learning)

Trong bài thực hành này, chúng ta sẽ làm quen các thuật giải cơ bản tiêu biểu cho hai phương pháp học giám sát và không giám sát.

1. GIỚI THIỆU

Học máy là quá trình xây dựng một mô hình dựa trên dữ liệu mẫu (dữ liệu quan sát), được gọi là dữ liệu huấn luyện, để đưa ra dự đoán hoặc quyết định mà không cần được lập trình chi tiết về việc đưa ra dự đoán hoặc quyết định này.

2. HỌC GIÁM SÁT

Là quá trình xây dựng mô hình học máy dựa trên tập dữ liệu đã được gán nhãn. Ví dụ, ta muốn dự báo thu nhập của một người với nhãn là $Y = \{\text{cao, thấp}\}$ dựa trên tập thuộc tính của người đó $X = \{\text{age, education, location}\}$. Mô hình sẽ học cách tính thu nhập của một người dựa trên tập X .

2.1. Mô hình dự báo Linear Regression

Mô hình này có tuổi đời hơn 200 tuổi. Tuy nhiên, đây là mô hình đơn giản và dễ hiểu. Trong phần thực hành về mô hình Linear Regression, chúng ta sẽ :

- Ước lượng thống kê dữ liệu huấn luyện
- Ước lượng hệ số cho mô hình Linear Regression
- Sử dụng Linear Regression để dự báo dữ liệu mới

Linear Regression xem như là một mối quan hệ phương trình đường thẳng giữa các biến đầu vào (X) với một biến đầu ra (Y)

$$y = b_0 + b_1 \times x$$

Hệ số b_0 và b_1 là hệ số mô hình có được từ tập huấn luyện.

Công việc chính là xác định được hệ số b_0 và b_1 bằng các hàm như mean, variance, hay covariance.

$$B1 = \text{sum}((x(i) - \text{mean}(x)) * (y(i) - \text{mean}(y))) / \text{sum}((x(i) - \text{mean}(x))^2)$$

$$B0 = \text{mean}(y) - B1 * \text{mean}(x)$$

Với i là giá trị thứ i của biến x

2.2. Dữ liệu huấn luyện Swedish Insurance

Bộ dữ liệu “Bảo hiểm ô tô ở Thụy Điển” để dự báo tổng số tiền bảo hiểm phải chi trả (Y) theo số lần yêu cầu (X).

X	Y
108	392,5
19	46,2
13	15,7
124	422,2
40	119,4
57	170,9

2.3. Tính Mean và Variance

- Hàm này ước lượng Mean và Variance cho biến x và y từ tập dữ liệu huấn luyện

$$\text{mean}(x) = \text{sum}(x) / \text{count}(x)$$

```
# Calculate the mean value of a list of numbers
def mean(values):
    return sum(values) / float(len(values))
```

- Variance được tính bằng tổng bình phương của hiệu mỗi giá trị x với Mean bằng công thức sau

$$\text{variance} = \text{sum}((x - \text{mean}(x))^2)$$

```
# Calculate the variance of a list of numbers
def variance(values, mean):
    return sum([(x-mean)**2 for x in values])
```

- Tính Mean và Variance cho tất cả X và Y

```
# Calculate the mean value of a list of numbers
def mean(values):
    return sum(values) / float(len(values))

# Calculate the variance of a list of numbers
def variance(values, mean):
    return sum([(x-mean)**2 for x in values])

# calculate mean and variance
dataset = [[1, 1], [2, 3], [4, 3], [3, 2], [5, 5]]
x = [row[0] for row in dataset]
y = [row[1] for row in dataset]
mean_x, mean_y = mean(x), mean(y)
var_x, var_y = variance(x, mean_x), variance(y, mean_y)
print('x stats: mean=%.3f variance=%.3f' % (mean_x, var_x))
print('y stats: mean=%.3f variance=%.3f' % (mean_y, var_y))
```

Yêu cầu: Sinh viên nhập mã và xem kết quả tính Mean, Variance

2.4. Tính Covariance

Covariance mô tả mối tương quan ảnh hưởng lẫn nhau của 2 nhóm X và Y. Hàm được tính theo công thức

$$\text{covariance} = \text{sum}((x(i) - \text{mean}(x)) * (y(i) - \text{mean}(y)))$$

```
# Calculate covariance between x and y
def covariance(x, mean_x, y, mean_y):
    covar = 0.0
    for i in range(len(x)):
        covar += (x[i] - mean_x) * (y[i] - mean_y)
    return covar
```

- Tính Mean và Covariance cho tập X và Y

```

# Calculate the mean value of a list of numbers
def mean(values):
    return sum(values) / float(len(values))

# Calculate the variance of a list of numbers
def variance(values, mean):
    return sum([(x-mean)**2 for x in values])

# Calculate covariance between x and y
def covariance(x, mean_x, y, mean_y):
    covar = 0.0
    for i in range(len(x)):
        covar += (x[i] - mean_x) * (y[i] - mean_y)
    return covar

# calculate covariance
dataset = [[1, 1], [2, 3], [4, 3], [3, 2], [5, 5]]
x = [row[0] for row in dataset]
y = [row[1] for row in dataset]
mean_x, mean_y = mean(x), mean(y)
covar = covariance(x, mean_x, y, mean_y)
print('Covariance: %.3f' % (covar))

```

Yêu cầu: Sinh viên nhập mã và xem kết quả Covariance

2.5. Ước lượng tham số B_0 và B_1 cho mô hình Linear Regression

$$B_0 = \text{mean}(y) - B_1 * \text{mean}(x)$$

$$B_1 = \text{covariance}(x, y) / \text{variance}(x)$$

Tính tham số B_0 và B_1

```

# Calculate the mean value of a list of numbers
def mean(values):
    return sum(values) / float(len(values))

# Calculate the variance of a list of numbers
def variance(values, mean):
    return sum([(x-mean)**2 for x in values])

# Calculate covariance between x and y
def covariance(x, mean_x, y, mean_y):
    covar = 0.0
    for i in range(len(x)):
        covar += (x[i] - mean_x) * (y[i] - mean_y)
    return covar

# Calculate coefficients
def coefficients(dataset):
    x = [row[0] for row in dataset]
    y = [row[1] for row in dataset]
    x_mean, y_mean = mean(x), mean(y)
    b1 = covariance(x, x_mean, y, y_mean) / variance(x, x_mean)
    b0 = y_mean - b1 * x_mean
    return [b0, b1]

# calculate coefficients
dataset = [[1, 1], [2, 3], [4, 3], [3, 2], [5, 5]]
b0, b1 = coefficients(dataset)
print('Coefficients: B0=%.3f, B1=%.3f' % (b0, b1))

```

Yêu cầu: Sinh viên nhập mã và xem kết quả tính hệ số B0, B1

2.6. Dự báo

Mô hình Linear Regression dự báo giá trị biến y_{new} cho giá trị x_{new} theo công thức

$$y_{\text{new}} = b0 + b1 * x_{\text{new}}$$

```

# Prediction
def simple_linear_regression(train, test):
    predictions = list()
    b0, b1 = coefficients(train)
    for row in test:
        yhat = b0 + b1 * row[0]
        predictions.append(yhat)
    return predictions

```

2.7. Hàm đánh giá giải thuật

Việc đánh giá giải thuật sẽ giúp cho chúng ta biết được mô hình học có hiệu quả hay không. Mô hình Linear Regression sẽ dùng hàm Root Mean Squared Error (RMSE) để đánh giá qua công thức sau

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Trong đó,

$\hat{y}_i$: là giá trị thứ i mà mô hình dự báo

y_i : là giá trị thật sự

n : số lượng quan sát

```
# Calculate root mean squared error
def rmse_metric(actual, predicted):
    sum_error = 0.0
    for i in range(len(actual)):
        prediction_error = predicted[i] - actual[i]
        sum_error += (prediction_error ** 2)
    mean_error = sum_error / float(len(actual))
    return sqrt(mean_error)

# Evaluate regression algorithm on training dataset
def evaluate_algorithm(dataset, algorithm):
    test_set = list()
    for row in dataset:
        row_copy = list(row)
        row_copy[-1] = None
        test_set.append(row_copy)
    predicted = algorithm(dataset, test_set)
    print(predicted)
    actual = [row[-1] for row in dataset]
    rmse = rmse_metric(actual, predicted)
    return rmse

dataset = [[1, 1], [2, 3], [4, 3], [3, 2], [5, 5]]
rmse = evaluate_algorithm(dataset, simple_linear_regression)
print('RMSE: %.3f' % (rmse))
```

Yêu cầu: sinh viên nhập mã trên và xem kết quả hàm đánh giá giải thuật

2.8. Cài đặt mô hình Linear Regression trên tập dữ liệu Swedish Insurance

2.8.1. Tải tập tin **SwedishInsurance.csv** do giảng viên cung cấp

2.8.2. Bổ sung hàm `load_csv` để đọc nội dung file `SwedishInsurance.csv`

```
# Load a CSV file
def load_csv(filename):
    dataset = list()
    with open(filename, 'r') as file:
        csv_reader = reader(file)
        next(csv_reader)
        for row in csv_reader:
            if not row:
                continue
            dataset.append(row)
    return dataset
```

2.8.3. Tách bộ dữ liệu thành 2 phần train (60%) và test (40%)

```
# Convert string column to float
def str_column_to_float(dataset, column):
    for row in dataset:
        row[column] = float(row[column].strip())

# Split a dataset into a train and test set
def train_test_split(dataset, split):
    train = list()
    train_size = split * len(dataset)
    dataset_copy = list(dataset)
    while len(train) < train_size:
        index = randrange(len(dataset_copy))
        train.append(dataset_copy.pop(index))
    return train, dataset_copy
```

2.8.4. Sửa lại hàm `evaluate_algorithm` để có thể áp dụng hàm `train_test_split` như sau

```
# Evaluate an algorithm using a train/test split
def evaluate_algorithm(dataset, algorithm, split, *args):
    train, test = train_test_split(dataset, split)
    test_set = list()
    for row in test:
        row_copy = list(row)
        row_copy[-1] = None
        test_set.append(row_copy)
    predicted = algorithm(train, test_set, *args)
    actual = [row[-1] for row in test]
    rmse = rmse_metric(actual, predicted)
    return rmse
```

2.8.5. Áp dụng giải thuật Linear Regression bằng cách thêm đoạn mã sau vào cuối

```
# Simple linear regression on insurance dataset
seed(1)
# load and prepare data
filename = 'SwedishInsurance.csv'
dataset = load_csv(filename)
for i in range(len(dataset[0])):
    str_column_to_float(dataset, i)
# evaluate algorithm
split = 0.6
rmse = evaluate_algorithm(dataset, simple_linear_regression, split)
print('RMSE: %.3f' % (rmse))
```

2.8.6. Để dự báo một giá trị mới x_{new} ta có thể nhập mã sau

```
b0, b1 = coefficients(dataset)
x_new = 18
y_new = b0 + b1 * x_new
print(y_new)
```

2.8.7. Trực quan hóa tập dữ liệu SwedishInsurance

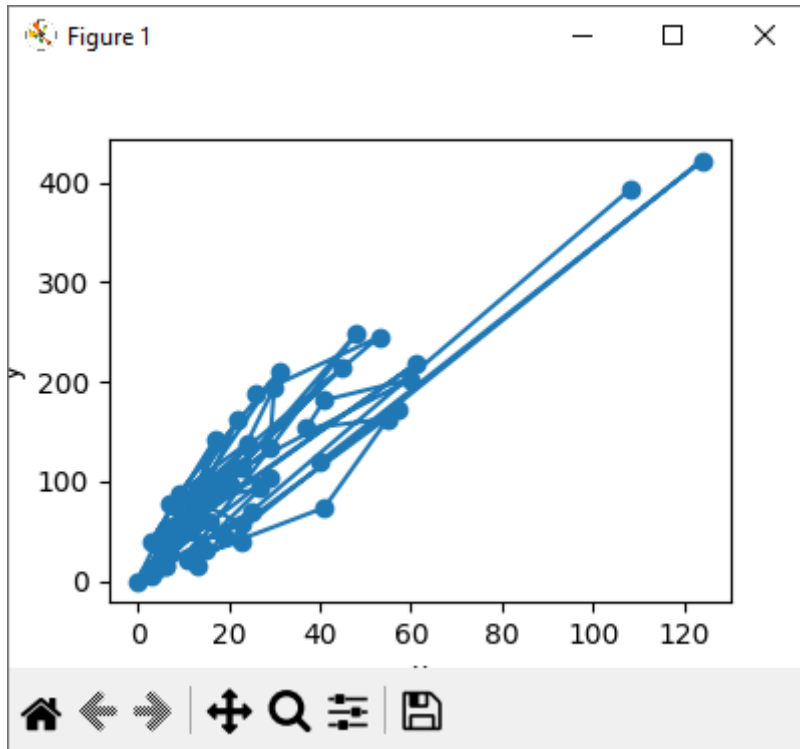
```
x, y = [], []
for row in dataset:
    x.append(row[0])
    y.append(row[1])

X = np.array(x)
Y = np.array(y)
plt.figure(figsize=(4, 3))
ax = plt.axes()
ax.scatter(x, y)
ax.plot(X, Y)
|
ax.set_xlabel('x')
ax.set_ylabel('y')

ax.axis('tight')

plt.show()
```

Kết quả



3. TÌM HIỂU FLASK

Flask là một Web framework viết bằng Python.

3.1. Cài đặt Flask

```
pip install Flask
```

3.2. Kiểm tra cài đặt Flask

Tạo mới 1 file và đặt tên Hello.py. Nhập đoạn mã sau

```
from flask import Flask
app = Flask(__name__)

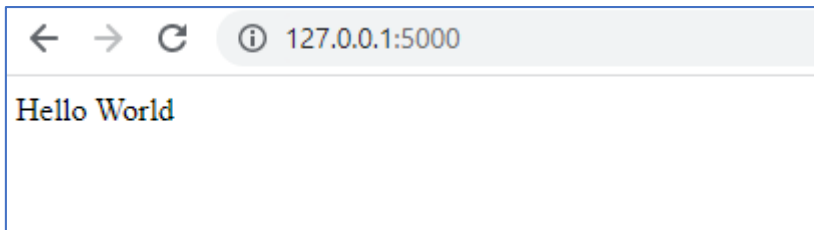
@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

Kết quả sau khi nhấn Run

```
* Serving Flask app 'Hello'
* Debug mode: off
[31m[1mWARNING: This is a development server. Do not use it in a production de
ployment. Use a production WSGI server instead.[0m
* Running on http://127.0.0.1:5000
[33mPress CTRL+C to quit[0m
|
```

Mở trình duyệt Web và gõ <http://127.0.0.1:5000>



3.3. Điều hướng trang mới với hàm Route()

Ta dùng hàm **route()** để điều hướng sang một trang mới

Nhập mã sau

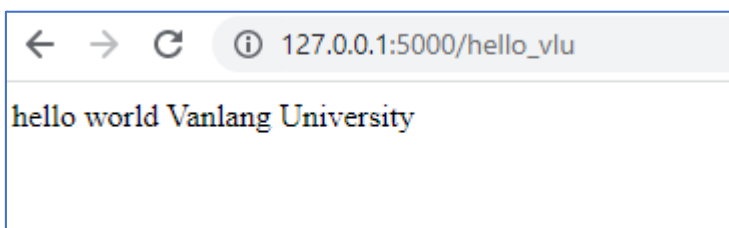
```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

@app.route('/hello_vlu')
def hello_world_vlu():
    return 'hello world Vanlang University'

if __name__ == '__main__':
    app.run()
```

Chạy Run code trên. Mở trình duyệt và gõ http://127.0.0.1:5000/hello_vlu



3.4. Truyền biến cho hàm qua URL

```
from flask import Flask
app = Flask(__name__)

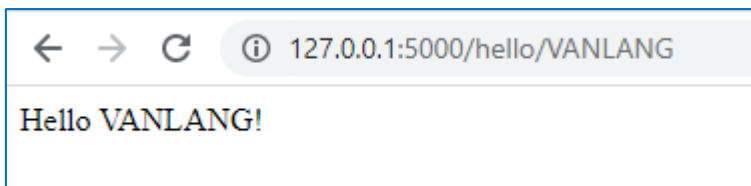
@app.route('/')
def hello_world():
    return 'Hello World'

@app.route('/hello_vlu')
def hello_world_vlu():
    return 'hello world Vanlang University'

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

if __name__ == '__main__':
    app.run()
```

Chọn Run code trên. Mở trình duyệt và gõ <http://127.0.0.1:5000/hello/VANLANG>



```

from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

@app.route('/hello_vlu')
def hello_world_vlu():
    return 'hello world Vanlang University'

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

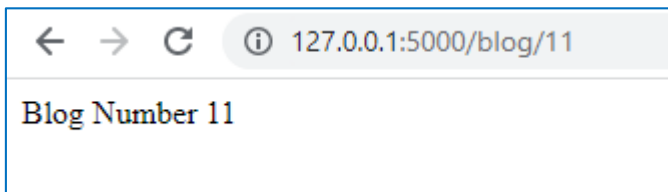
@app.route('/blog/<int:postID>')
def show_blog(postID):
    return 'Blog Number %d' % postID

@app.route('/rev/<float:revNo>')
def revision(revNo):
    return 'Revision Number %f' % revNo

if __name__ == '__main__':
    app.run()

```

Chọn Run code trên. Mở trình duyệt và gõ <http://127.0.0.1:5000/blog/11>



3.5. Điều hướng Web với hàm url_for()

Tương tự hàm **route()** nhưng khác ở chỗ cho phép truyền tên hàm và giá trị tham số hàm

```

from flask import Flask, redirect, url_for
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

@app.route('/hello_vlu')
def hello_world_vlu():
    return 'hello world Vanlang University'

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

@app.route('/blog/<int:postID>')
def show_blog(postID):
    return 'Blog Number %d' % postID

@app.route('/rev/<float:revNo>')
def revision(revNo):
    return 'Revision Number %f' % revNo

@app.route('/admin')
def hello_admin():
    return 'Hello Admin'

@app.route('/guest/<guest>')
def hello_guest(guest):
    return 'Hello %s as Guest' % guest

@app.route('/user/<name>')
def hello_user(name):
    if name == 'admin':
        return redirect(url_for('hello_admin'))
    else:
        return redirect(url_for('hello_guest', guest = name))

if __name__ == '__main__':
    app.run()

```

Chọn Run code trên. Mở trình duyệt và gõ <http://127.0.0.1:5000/user/admin>

4. BÀI TẬP

1. Sử dụng bộ dữ liệu titanic.csv và viết chương trình dự báo giá tiền một hành khách phải trả theo độ tuổi của họ
2. Sử dụng bộ dữ liệu Fish.csv và viết chương trình dự báo chiều rộng của 1 giống cá theo trọng lượng. Cho biết thêm giống cá đó là giống cá gì?
3. Sử dụng Flask để triển khai các câu 1 - 4 trên nền tảng Web.